Observer Design Pattern (Behavioral)

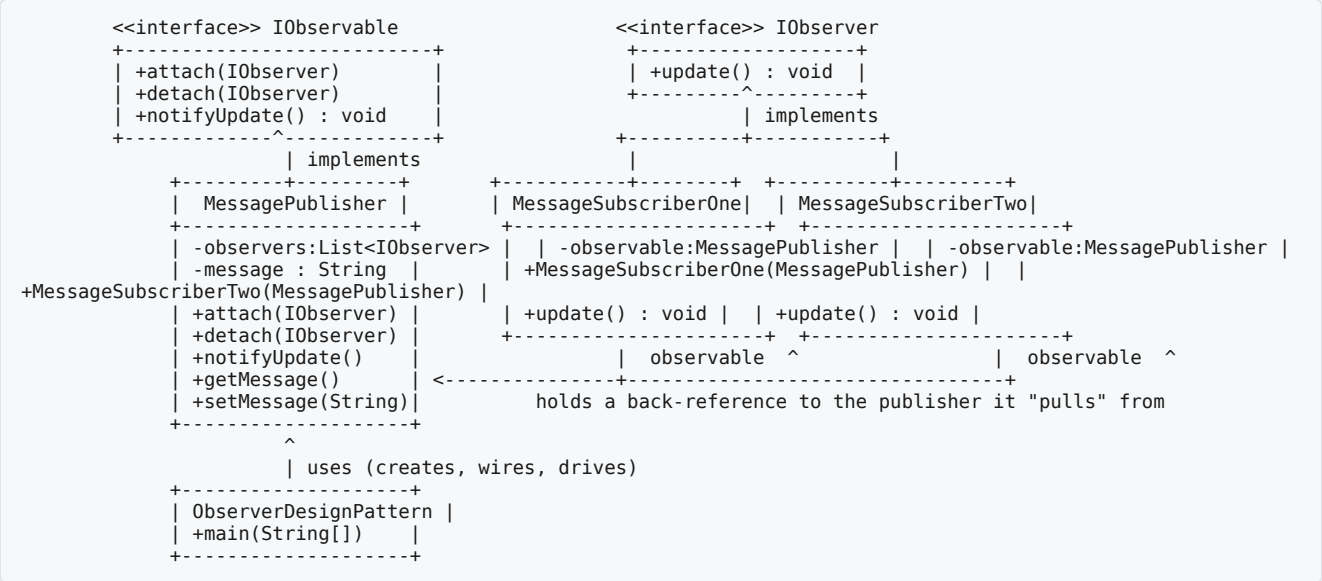
Define a one-to-many dependency between objects so that when one object changes state, all of its dependents are notified and updated automatically.

Intent

Define a **one-to-many** dependency between objects so that when one object (the **subject / observable**) changes state, **all** of its dependents (the **observers**) are notified and updated **automatically** — without the subject ever knowing the concrete type of any observer. This is the classic "publish/subscribe" relationship: subscribers register themselves with a publisher, and from that point on the publisher broadcasts every state change to whoever is currently listening, and only to the shared `IObserver` contract — never to a named subscriber class.

In this module the subject is a **`MessagePublisher`** that holds a message, and the observers are **subscribers** that print the message whenever it changes.

UML class diagram (ASCII)



- `MessagePublisher` depends only on the `IObserver` **interface** — the upward arrow from `MessagePublisher` to `IObservable` is an *implements* relation; its `List<IObserver> observers` field is drawn against the interface, not either concrete subscriber.
- Both subscribers depend on the **concrete** `MessagePublisher` (not just `IObservable`) because they need `getMessage()`, which is not part of the `IObservable` contract — this is the "pull" half of the relationship, explained below.

The players

```
observer/IObserver                                the observer contract: update()
observer/concreteObserver/MessageSubscriberOne    concrete observer – prints the message on update()
                         /MessageSubscriberTwo    concrete observer – prints the message on update()

subject/IObservable                               the subject contract: attach / detach / notifyUpdate
subject/concreteSubject/MessagePublisher          concrete subject – keeps the observer list + the message

ObserverDesignPattern                             the demo – wires a publisher and two subscribers, drives
state changes
```

Two contracts facing each other:

- **`IObservable`** — the publisher side of the relationship (register / unregister / broadcast).
- **`IObserver`** — the subscriber side of the relationship (be notified).

Code walkthrough — every line explained

`IObserver.java` — the subscriber contract

```
package com.design.patterns.observer;

public interface IObserver {
    public void update();
}
```

- `package com.design.patterns.observer;` — Places the observer-side contract in its own `observer` sub-package, separate from the subject side. Keeping the two contracts in different packages makes the "two interfaces facing each other" structure visible in the directory layout, not just in the code.
- `public interface IObserver {` — Declares a `public` interface named `IObserver`. An interface (rather than an abstract class) is used because a subscriber's *only* obligation to the pattern is "be reachable at `update()`" — there is no shared state or behavior to inherit, so a pure contract is the right and smallest tool. `public` lets any class in any package implement it. The `{` opens the interface body.
- `public void update();` — Declares the single abstract method every observer must implement. It takes **no arguments** and returns **no value**: it is a pure notification — "something changed" — not a delivery of data. This is the **pull model** (see *Why* below): the method only tells the observer *that* a change happened; the observer must ask the subject for details itself. The `public` modifier is redundant on an interface method (interface members are implicitly `public abstract`) but is written explicitly here for clarity.
- `}` — Closes the `IObserver` interface body.

`IObservable.java` — the publisher contract

```
package com.design.patterns.subject;

import com.design.patterns.observer.IObserver;

public interface IObservable {
    public void attach(IObserver observer);

    public void detach(IObserver observer);

    public void notifyUpdate();

}
```

- `package com.design.patterns.subject;` — Places the subject-side contract in its own `subject` package, mirroring `observer`.
- `import com.design.patterns.observer.IObserver;` — Brings the `IObserver` type from the sibling package into scope, because `attach` / `detach` need to name it in their signatures.
- `public interface IObservable {` — Declares the subject's contract as an interface for the same reason as `IObserver`: it is pure behavior with no shared state to inherit. The `{` opens the interface body.
- `public void attach(IObserver observer);` — **Subscribe.** Declares the method a subject exposes to add an observer to its notification list. It accepts the `IObserver` interface type, not any concrete subscriber class — this is what lets the subject stay ignorant of *which* subscriber classes exist.
- `public void detach(IObserver observer);` — **Unsubscribe.** The mirror operation: remove a previously attached observer so it stops being notified. Without this, a long-lived subject would keep references to (and keep notifying) observers that no longer care — the classic **lapsed-listener** leak.
- `public void notifyUpdate();` — **Broadcast.** Declares the method that tells *every currently attached* observer to react. This is the method that actually drives the one-to-many fan-out that is the whole point of the pattern.
- `}` — Closes the `IObservable` interface body.

`MessagePublisher.java` — the concrete subject

```
package com.design.patterns.subject.concreteSubject;

import java.util.ArrayList;
import java.util.List;

import com.design.patterns.observer.IObserver;
import com.design.patterns.subject.IObservable;

public class MessagePublisher implements IObservable {

    private List<IObserver> observers = new ArrayList<>();

    private String message;

    @Override
    public void attach(IObserver observer) {
        observers.add(observer);
```

```
    }

    @Override
    public void detach(IObserver observer) {
        observers.remove(observer);
    }

    @Override
    public void notifyUpdate() {
        observers.forEach(IObserver::update);
    }

    public String getMessage() {
        return message;
    }

    public void setMessage(String message) {
        this.message = message;
        notifyUpdate();
    }
}
```

- `package com.design.patterns.subject.concreteSubject;` — The concrete subject lives one level below `subject`, in a dedicated `concreteSubject` sub-package. This separates the *contract* (`subject.IObservable`) from its *implementation*, so the module can (in principle) hold more than one concrete subject without crowding the `subject` package.
- `import java.util.ArrayList;` / `import java.util.List;` — Bring in the JDK collection types used to hold the list of registered observers.
- `import com.design.patterns.observer.IObserver;` — Needed because the observer list is typed against the `IObserver` interface, not a concrete subscriber.
- `import com.design.patterns.subject.IObservable;` — Needed to implement the `IObservable` contract.
- `public class MessagePublisher implements IObservable {` — Declares the concrete subject. `implements IObservable` is the load-bearing part of the pattern: it forces `MessagePublisher` to provide `attach`, `detach`, and `notifyUpdate`, and lets every caller depend on the interface instead of this class. The `{` opens the class body.
- `private List<IObserver> observers = new ArrayList<>();` — The subscriber registry, declared **against the interface type** `IObserver`, not `MessageSubscriberOne` / `MessageSubscriberTwo`. This is what makes the subject's broadcast logic (`notifyUpdate`) work for *any* current or future observer type without ever being edited. `private` keeps the list itself encapsulated — only this class can add or remove entries, and only through `attach` / `detach`. `ArrayList` gives ordered, dynamically-resizable storage; observers can be added/removed at any time at runtime.
- `private String message;` — The one piece of subject state that observers care about. It starts `null` until `setMessage` is called.
- `@Override public void attach(IObserver observer) { observers.add(observer); }` — Implements subscribe: appends the given observer to the registry. `@Override` documents (and lets the compiler check) that this method satisfies the `IObservable` contract. The body is intentionally the simplest possible operation — add to a list — because the whole point is that the subject doesn't need to do anything type-specific for any observer.
- `@Override public void detach(IObserver observer) { observers.remove(observer); }` — Implements unsubscribe: removes the observer from the registry (`List.remove(Object)` compares by `equals`, which for these subscriber classes falls back to reference identity, so this removes the exact instance that was attached). After this call, that observer will no longer be reached by `notifyUpdate()`.
- `@Override public void notifyUpdate() { observers.forEach(IObserver::update); }` — Implements broadcast: walks the **current** observer list and invokes `update()` on each entry via the method reference `IObserver::update`. This line is the crux of Observer: one call fans out to every attached listener, and the loop body only ever calls the interface method `update()` — never anything specific to `MessageSubscriberOne` or `Two`. This is also where the earlier `detach` takes effect: an observer that was removed simply isn't in `observers` any more, so it is skipped here.
- `public String getMessage() { return message; }` — Exposes the current message so an observer can **pull** it after being told "something changed." This is how the pull model closes the loop: `update()` carries no data, so the observer calls back into the subject to fetch what it needs.
- `public void setMessage(String message) { this.message = message; notifyUpdate(); }` — The state-changing operation. It stores the new value in the field (shadowed by the parameter of the same name, disambiguated with `this.`), then **immediately calls `notifyUpdate()`**. This one line is what makes the class a true Observer subject rather than a passive data holder: every state change is automatically, synchronously broadcast to every attached observer — callers never have to remember to notify anyone themselves.
- `}` — Closes the `MessagePublisher` class body.

`MessageSubscriberOne.java` / `MessageSubscriberTwo.java` — the concrete observers

```
package com.design.patterns.observer.concreteObserver;

import com.design.patterns.observer.IObserver;
import com.design.patterns.subject.concreteSubject.MessagePublisher;

public class MessageSubscriberOne implements IObserver {

    private MessagePublisher observable;

    public MessageSubscriberOne(MessagePublisher observable) {
```

```
        this.observable = observable;
        this.observable.attach(this);
    }

    @Override
    public void update() {
        System.out.println("SubscriberOne received: " + this.observable.getMessage());
    }

}
```

- `package com.design.patterns.observer.concreteObserver;` — The concrete observers live one level below `observer`, mirroring `subject.concreteSubject` — contract and implementation are split the same way on both sides of the pattern.
- `import com.design.patterns.observer.IObserver;` — Needed to `implement IObserver`.
- `import com.design.patterns.subject.concreteSubject.MessagePublisher;` — Needed because this observer holds a reference to the **concrete** publisher (not just `IObservable`) — explained on the field below.
- `public class MessageSubscriberOne implements IObserver {` — Declares the concrete observer. `implements IObserver` is what lets `MessagePublisher.notifyUpdate()` call this class through the shared interface. The `{` opens the class body.
- `private MessagePublisher observable;` — A back-reference to the publisher this subscriber is watching. It is typed as the **concrete** `MessagePublisher`, not the `IObservable` interface, because this class needs `getMessage()` in `update()` below, and `getMessage()` is not part of the `IObservable` contract (only `attach` / `detach` / `notifyUpdate` are). `private` keeps the reference encapsulated inside the subscriber.
- `public MessageSubscriberOne(MessagePublisher observable) {` — The constructor takes the publisher to subscribe to as its only argument.
- `this.observable = observable;` — Stores the publisher reference on the field (the parameter shadows the field, disambiguated with `this.`), so `update()` can use it later.
- `this.observable.attach(this);` — **Self-registration.** The subscriber attaches *itself* to the publisher's observer list at construction time, passing `this` (which satisfies `IObserver` because the class `implements IObserver`). The practical effect: the instant a `MessageSubscriberOne` is constructed, it is already listening — there is no separate "now go subscribe" step a caller could forget.
- `}` — Closes the constructor body.
- `@Override public void update() {` — Implements the `IObserver` contract's single method — this is what `MessagePublisher.notifyUpdate()` calls on every attached observer.
- `System.out.println("SubscriberOne received: " + this.observable.getMessage());` — The reaction: pull the publisher's *current* message via `getMessage()` and print it, prefixed with this subscriber's own label so the console output shows which subscriber reacted. Because the value is pulled fresh from `observable` at the moment `update()` runs, it always reflects the latest state, not whatever the message was when this subscriber was constructed.
- `}` — Closes the `update()` method body.
- `}` — Closes the `MessageSubscriberOne` class body.

`MessageSubscriberTwo.java` is structurally identical, differing only in its class name, constructor name, and the `"SubscriberTwo received: "` print prefix. Having two independent subscribers demonstrates that a single state change fans out to an open-ended number of listeners, each reacting independently.

`ObserverDesignPattern.java` — the demo / driver

```
package com.design.patterns;

import com.design.patterns.observer.concreteObserver.MessageSubscriberOne;
import com.design.patterns.observer.concreteObserver.MessageSubscriberTwo;
import com.design.patterns.subject.concreteSubject.MessagePublisher;

public class ObserverDesignPattern {

    public static void main(String[] args) {
        MessagePublisher publisher = new MessagePublisher();

        MessageSubscriberOne subscriberOne = new MessageSubscriberOne(publisher);
        MessageSubscriberTwo subscriberTwo = new MessageSubscriberTwo(publisher);

        publisher.setMessage("first message");

        publisher.detach(subscriberOne);

        publisher.setMessage("second message");
    }

}
```

- `package com.design.patterns;` — The driver sits in the top-level module package, one level above both `observer` and `subject`, since it is the client that wires the two sides together rather than belonging to either.
- The three `import` statements bring the concrete subject and both concrete observers into scope so they can be referred to by simple name.
- `public class ObserverDesignPattern {` — Declares the module's entry-point class. `public` is required so the JVM launcher can resolve it by name from the command line. The filename `ObserverDesignPattern.java` matches this

class name exactly, as Java requires for a public top-level type. The `{` opens the class body.
- `public static void main(String[] args) {` — The standard Java entry point: `public` so the JVM can call it, `static` so it runs without an instance, `void` because it returns nothing, `String[] args` for (unused) command-line arguments.
- `MessagePublisher publisher = new MessagePublisher();` — Creates the subject. At this point `observers` is empty and `message` is `null`.
- `MessageSubscriberOne subscriberOne = new MessageSubscriberOne(publisher);` — Constructs the first subscriber, passing `publisher`. Its constructor immediately calls `publisher.attach(this)`, so after this line `publisher`'s observer list is `[subscriberOne]`.
- `MessageSubscriberTwo subscriberTwo = new MessageSubscriberTwo(publisher);` — Constructs the second subscriber the same way. After this line the observer list is `[subscriberOne, subscriberTwo]`.
- `publisher.setMessage("first message");` — Sets the message and, because `setMessage` ends with `notifyUpdate()`, immediately broadcasts to **every** attached observer: both `subscriberOne` and `subscriberTwo` fire `update()` and print the message.
- `publisher.detach(subscriberOne);` — Unsubscribes `subscriberOne`. The observer list is now `[subscriberTwo]`; `subscriberOne` will no longer be notified of anything.
- `publisher.setMessage("second message");` — Sets a new message and broadcasts again. Because `subscriberOne` was detached, only `subscriberTwo` is in the list and only it fires `update()` — this line demonstrates that `detach` genuinely removes an observer from future broadcasts, not just that it *could*.
- `}` — Closes the `main` method body.
- `}` — Closes the `ObserverDesignPattern` class body.

Why these design decisions

Why is the subject/observer relationship expressed as two interfaces instead of the publisher calling subscribers directly? Without interfaces, `MessagePublisher` would need a hard-coded reference to every subscriber class and would call each one by name — tightly coupling the subject to specific classes and forcing a code change (and a recompile of the subject) every time a new subscriber type appears. Observer inverts this: subscribers register themselves against a shared `IObserver` contract, and the subject just walks an anonymous list calling one method. New subscriber types can be added with **zero changes** to `MessagePublisher` or `IObservable`.

Why does the subject hold `List<IObserver>` (the interface), not a list of concrete subscribers? So the subject is decoupled from *who* is listening. `notifyUpdate()` can call `update()` on anything that implements `IObserver` — present or future — without an `instanceof` check or a cast anywhere in `MessagePublisher`. This is "program to an interface, not an implementation" applied directly to the broadcast list.

Why do observers call `attach(this)` from their own constructor, instead of the caller attaching them from the outside? It makes subscription automatic and atomic: the moment a `MessageSubscriberOne`/`Two` object exists, it is already on the publisher's notify list — there is no separate step a caller could forget. The trade-off is that it couples an observer instance to the one publisher passed into its constructor, and it leaks `this` out of a constructor before the object is fully "handed off," which is worth being careful about in multithreaded code. A common alternative is to attach explicitly from the outside (`publisher.attach(subscriber)`); this module intentionally uses self-registration to make "constructed ⇒ subscribed" a hard guarantee.

Why the "pull" model (`getMessage()`) instead of "push" (passing the new value into `update(...)`)? There are two conventional flavors of Observer: - **Push** — the subject passes the changed data as an argument, e.g. `update(String newMessage)`. - **Pull (this module)** — the subject just announces "something changed" via a no-argument `update()`, and the observer **pulls** whatever it needs straight from the subject (`observable.getMessage()`).

Pull keeps the `IObserver` interface tiny and generic — one no-argument method works no matter what kind of state changed or how many fields are involved — and lets each observer decide exactly which parts of the subject's state it cares about. The cost is that every observer must hold a reference back to the subject to pull from it, which is exactly why `MessageSubscriberOne`/`Two` store `observable`.

Why does `setMessage` call `notifyUpdate()` itself, instead of leaving the caller to call it separately? So that "state changed" and "observers were told" can never drift apart. If the caller were responsible for calling `notifyUpdate()` after `setMessage`, it would be possible to change the message and forget to broadcast it (or vice versa). Folding the notification into the setter makes the subject side of the contract self-enforcing — every state change is guaranteed to reach every current observer.

Why keep `detach` even though it's easy to write a demo that never calls it? Lifecycle management. Real observers come and go; `detach` lets a subscriber stop listening so the subject doesn't keep notifying — and keep alive — observers that no longer care. This module's `main` deliberately exercises `detach(subscriberOne)` before the second `setMessage` specifically so the demo output proves that detaching actually removes an observer from future broadcasts (see *Execution flow* below). Omitting `detach` in long-running systems is a classic source of the **lapsed-listener memory leak**.

Execution flow (trace of `main`)

```
main
 |
```

```
├── new MessagePublisher()                          observers = []                      message = null
│
├── new MessageSubscriberOne(publisher)
│       └── publisher.attach(this)                  observers = [One]
│
├── new MessageSubscriberTwo(publisher)
│       └── publisher.attach(this)                  observers = [One, Two]
│
├── publisher.setMessage("first message")           message = "first message"
│       └── notifyUpdate()
│               ├── One.update()  → prints "SubscriberOne received: first message"
│               └── Two.update()  → prints "SubscriberTwo received: first message"
│
├── publisher.detach(subscriberOne)                  observers = [Two]
│
└── publisher.setMessage("second message")          message = "second message"
        └── notifyUpdate()
                └── Two.update()  → prints "SubscriberTwo received: second message"
                    (One is no longer in observers, so it is never called)
```

Every state change flows through exactly one path — `setMessage → notifyUpdate → observer.update()` for each currently-attached observer — with no manual, per-subscriber calls anywhere in the demo. That single path is what makes this a textbook (not merely partial) Observer implementation.

Expected output

Captured verbatim from a real run (see *How to run*):

```
SubscriberOne received: first message
SubscriberTwo received: first message
SubscriberTwo received: second message
```

- Both subscribers print for `"first message"` because both are attached when it is set.
- Only `SubscriberTwo` prints for `"second message"` because `subscriberOne` was detached beforehand.

How to run

From inside this module's own directory (this module is part of a larger multi-module reactor; build it standalone, not from the reactor root):

```
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 mvn -o clean compile
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 java -cp target/classes com.design.patterns.ObserverDesignPattern
```

(`-o` runs Maven offline against the local `~/.m2` cache; drop it if dependency resolution needs the network. JDK 11 is required in this environment because the reactor's default JDK breaks a Lombok-using sibling module — this module itself doesn't use Lombok or Spring at runtime.)

Relationship to the other Behavioral patterns

- **Observer (this module)** — one subject broadcasts to **many** observers that registered themselves; a single state change fans out to every current listener.
- **Chain of Responsibility** — a request travels to **one** handler along a chain, not broadcast to all.
- **Mediator** — centralizes many-to-many communication in a hub, instead of subjects notifying observers directly.
- **State / Strategy** — a context delegates to a *single* swapped-in object, not a list of listeners.

Reach for Observer when a change in one object must be reflected in an **open-ended set** of others, and the source object should stay ignorant of who (or how many) is listening — event systems, UI data-binding, and pub/sub messaging are the canonical uses.